

Eventara - Solutions Blueprint & Remediation Roadmap

From prototype to production, on a shoestring. A concrete, cost-first plan that fixes every issue raised in the Product Audit while **reusing the existing front-end** wherever possible.

Companion to	Eventara Product & Engineering Due-Diligence Audit
Audience	Founders / product+eng team (small, budget-constrained)
Guiding rule	Free tier first. Add paid only when volume forces it. Keep the front-end; add a backend behind it.
Current stack	Static HTML/CSS/JS (13 pages), one <code>styles.css</code> , <code>auth.js</code> (localStorage), Vercel Hobby, one Gemini serverless fn
Date	22 July 2026

The one idea that makes this cheap: the current front-end already funnels all auth through a **single module** (`window.Auth` in `auth.js`) and renders data from hard-coded HTML. That means we can **swap a real backend in behind the same interface** without rewriting pages. We keep the design system, the pages, and the UX - we replace the *plumbing*.

1. Recommended architecture (free-tier, incremental)

We do **not** rebuild. We add a thin backend and progressively replace mock data with live data. One platform - **Supabase** - collapses five of the audit's "Critical" issues (database, server auth, authorization, storage, real data) into a single free service.

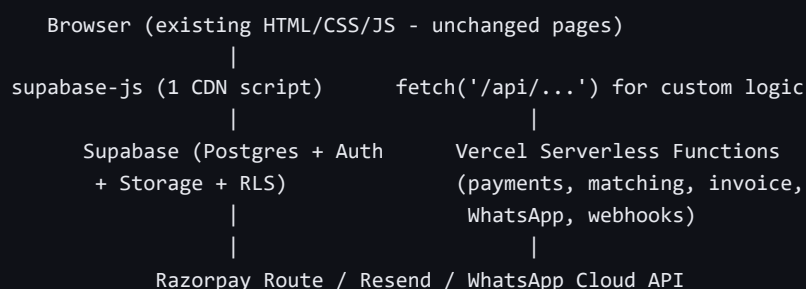
1.1 The target stack

Layer	Recommended (free tier)	Why this one	Free limits (enough for pilot)
Database	Supabase Postgres	Real Postgres + auto REST/GraphQL + Realtime + Row-Level Security, all managed	500 MB DB, 2 projects, unlimited API requests
Auth	Supabase Auth	Email OTP / magic-link / phone OTP, JWT sessions, social logins - replaces the insecure localStorage guard	50,000 monthly active users free
File storage	Supabase Storage (or Cloudinary)	Owned, rights-clean supplier media + dispute evidence; S3-compatible	1 GB storage, 5 GB egress (Cloudinary: 25 credits)
Server logic	Vercel Serverless Functions (already used)	Payments webhooks, matching, invoice PDF, WhatsApp send - you already ship <code>api/chat.js</code>	100 GB-hrs, generous on Hobby
Payments + escrow	Razorpay Route (or Cashfree Easy-Split)	RBI-compliant marketplace split/escrow; no monthly fee, pay-per-txn	No setup/monthly fee; ~2% per transaction
Email	Resend (or Brevo)	Transactional email (quotes, confirmations, invoices)	3,000 emails/mo, 100/day

Layer	Recommended (free tier)	Why this one	Free limits (enough for pilot)
WhatsApp	Meta WhatsApp Cloud API (or Gupshup)	India runs on WhatsApp; SLA + booking + payout alerts	~1,000 free service conversations/mo
Analytics	PostHog Cloud	Funnels, events, session replay - see where the funnel leaks	1M events/mo free
Errors	Sentry	Catch JS + serverless exceptions in the wild	5,000 errors/mo free
Front-end build	Astro (static output)	Componentize nav/footer, hashed assets, keep HTML-first; deploys to Vercel	Free (open source)
Hosting	Vercel Hobby (already)	Same as today; Astro + serverless fns fit natively	100 GB bandwidth
Search (later)	Postgres full-text / Typesense Cloud free trial or Meilisearch self-host	Start with Postgres FTS; upgrade only if needed	Postgres FTS = free

Total cash to reach a working pilot: Rs 0/month (only transaction fees when real money moves). At a small commercial scale (say 200 bookings/mo) expect roughly **Rs 0-2,000/month** in tooling plus the ~2% payment fee, which is passed through the take-rate.

1.2 How it plugs into the current site (no rewrite)



- **Keep** all 13 pages, `styles.css`, and the component look-and-feel.
- **Rewrite the internals** of `auth.js` to call Supabase Auth while **preserving its public API** (`Auth.login`, `Auth.getSession`, `Auth.requireRole`, `Auth.renderNav`) - so every page that already calls those keeps working.
- **Replace hard-coded HTML data** (search results, dashboard tables) with `supabase.from('...').select()` queries, one page at a time.

2. Foundation setup (do this first - ~1 week, Rs 0)

These steps unblock ~8 of the audit's Critical issues at once.

Step 1 - Create the backbone accounts (all free): Supabase project, Vercel (exists), Razorpay test account, Resend, PostHog, Sentry, a GitHub repo made **private** (fixes the public-repo finding).

Step 2 - Add the Supabase client to the site. In the shared `<head>` (or the new Astro layout), add:

```
<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script>
  window.sb = supabase.createClient(
    'https://YOUR-PROJECT.supabase.co',
    'YOUR-ANON-PUBLIC-KEY' // safe to expose; RLS protects the data
  );
</script>
```

Step 3 - Create the schema (SQL in the Supabase editor). Minimum viable tables:

```

-- profiles: 1:1 with auth.users; role decides customer vs supplier
create table profiles (
  id uuid primary key references auth.users on delete cascade,
  role text not null check (role in ('customer','supplier')),
  name text, phone text, city text default 'Udaipur',
  created_at timestamptz default now()
);

create table suppliers (
  id uuid primary key default gen_random_uuid(),
  owner uuid references profiles(id),
  business_name text not null, category text, description text,
  gstin text, pan text, fssai text,
  verified boolean default false,          -- set only by the KYC pipeline
  rating numeric default 0, review_count int default 0,
  capacity int, starting_price int, cancellation_policy text,
  created_at timestamptz default now()
);

create table listings (          -- a bookable space/package
  id uuid primary key default gen_random_uuid(),
  supplier uuid references suppliers(id),
  title text, price_from int, amenities text[], media text[]
);

create table availability (      -- single source of truth for dates
  supplier uuid references suppliers(id),
  day date not null,
  state text not null check (state in ('open','blocked','maintenance','held','booked')),
  primary key (supplier, day)
);

create table enquiries (
  id uuid primary key default gen_random_uuid(),
  customer uuid references profiles(id),
  event_type text, event_date date, guests int, budget_band text,
  status text default 'new', created_at timestamptz default now()
);

create table quotes (
  id uuid primary key default gen_random_uuid(),
  enquiry uuid references enquiries(id),
  supplier uuid references suppliers(id),
  line_items jsonb, total int, status text default 'submitted',
  created_at timestamptz default now()
);

create table bookings (
  id uuid primary key default gen_random_uuid(),
  quote uuid references quotes(id),
  customer uuid references profiles(id),
  supplier uuid references suppliers(id),
  amount int, deposit int, status text default 'confirmed',
  payment_ref text, created_at timestamptz default now()
);

create table reviews (
  id uuid primary key default gen_random_uuid(),
  booking uuid references bookings(id) unique,  -- only 1 review per real booking
  customer uuid references profiles(id),
  supplier uuid references suppliers(id),
  rating int check (rating between 1 and 5), comment text,

```

```

    created_at timestamptz default now()
  );

create table disputes (
  id uuid primary key default gen_random_uuid(),
  booking uuid references bookings(id),
  raised_by uuid references profiles(id),
  type text, priority text, status text default 'open',
  evidence text[], created_at timestamptz default now()
);

create table notifications (
  id uuid primary key default gen_random_uuid(),
  recipient uuid references profiles(id),
  kind text, title text, body text,
  read boolean default false, created_at timestamptz default now()
);

```

Step 4 - Turn on Row-Level Security (this is the real access control). Example:

```

alter table bookings enable row level security;

-- a customer sees only their bookings; a supplier only theirs
create policy "own bookings (customer)" on bookings
  for select using (auth.uid() = customer);
create policy "own bookings (supplier)" on bookings
  for select using (auth.uid() in (select owner from suppliers where id = supplier));

```

With RLS on, the front-end can query Postgres directly and **the database enforces who sees what** - this is what makes the "client can bypass auth" issue go away, because bypassing the client no longer grants data access.

3. Solutions for every audit issue

Each row: the fix, the concrete steps, the free tool, effort, and which roadmap phase it lands in. Effort is a rough team-week estimate for a 1-2 engineer team.

3.1 Critical issues

#	Audit issue	Solution & key steps	Free tool	Effort	Phase
1	No backend / no real transactions	Stand up Supabase (Sec. 2). Create schema. Point one flow (booking) at real tables end to end.	Supabase	1-2 wk	1
2	Client-side auth bypassable	Rewrite <code>auth.js</code> internals to Supabase Auth (email/phone OTP), JWT in httpOnly cookie; enforce RLS on every table; keep the same <code>Auth.*</code> API so pages are untouched.	Supabase Auth	1 wk	1
3	Liquidity / cold-start	Concierge supply first: hand-onboard 30-50 Udaipur venues into <code>suppliers</code> ; run demand manually; only then open self-serve. Track fill rate.	Supabase + humans	ongoing	1-2
4	No reviews	<code>reviews</code> table with a unique constraint on booking_id (only real, completed bookings can review); recompute <code>suppliers.rating/review_count</code> via a Postgres trigger; render on listing + profile.	Supabase	1 wk	2

#	Audit issue	Solution & key steps	Free tool	Effort	Phase
5	Search doesn't search	Phase A: Postgres full-text + filter query behind the existing search UI. Phase B: Typesense/Meilisearch if needed. Availability-aware via a join to <code>availability</code> .	Postgres FTS	1-2 wk	2
6	Trust badges fake	KYC pipeline: capture GSTIN/PAN/FSSAI + docs to Storage; a serverless fn validates GSTIN (govt/3rd-party API) and sets <code>suppliers.verified=true</code> ; badge reads the real column.	Vercel fn + Storage	1-2 wk	2
7	Escrow fictional	Razorpay Route (or Cashfree Easy-Split): customer pays into a nodal account; funds held; released to supplier on delivery-confirm or auto after T+72h. Serverless fn creates orders; webhook updates <code>bookings</code> /ledger.	Razorpay Route	2-3 wk	1-2
8	No persistence	Every form (<code>brief</code> , profile, settings, disputes, help) does <code>sb.from('...').insert()</code> ; replace toasts with real writes + optimistic UI.	Supabase	ongoing	1
9	Hard-coded creds / single tenant	Delete demo creds from source; real users in <code>auth.users</code> ; supplier context from the session's <code>suppliers</code> row, not a hard-coded "Paandora Grand".	Supabase Auth	3 days	1
10	Duplicated markup (13 files)	Migrate to Astro : one <code>Layout.astro</code> + <code><Navbar/></code> <code><Footer/></code> components; paste existing HTML into components almost verbatim. One nav change = one file.	Astro	1 wk	0-1
11	Manual <code>?v=N</code> cache-busting	Astro/Vite emits <code>content-hashed</code> filenames automatically; delete the manual bumping ritual entirely.	Astro/Vite	included	0-1
12	No availability sync	The <code>availability</code> table is the single source of truth; search filters on it, quote holds a date (<code>held</code>), booking flips it to <code>booked</code> . Realtime keeps the supplier calendar live.	Supabase	1 wk	2
13	Dead (toast-only) buttons	Wire the 5 golden actions to real endpoints first (Build Quote, Accept, Pay Deposit, Confirm Delivery, Raise Dispute); keep toasts only as success feedback.	Supabase/Vercel	2 wk	1-2
14	No notifications backbone	A <code>notifications</code> table + a serverless "notify" fn that also sends WhatsApp (Cloud API) and email (Resend). Triggers on enquiry, quote, booking, payout, dispute.	WhatsApp + Resend	1-2 wk	2
15	Thin moat	Build proprietary data: verified reviews, on-time-delivery track record, exclusive supply contracts. Moat = network + data, not features.	(strategy)	ongoing	2-3
16	Small TAM / long cycle	Prove Udaipur unit economics first; pre-write the expansion playbook (adjacent cities OR the wedding vertical) with a decision gate on repeat-rate + margin.	(strategy)	ongoing	3
17	No analytics	Add PostHog snippet to the layout; instrument the funnel (search->brief->quote_view->booking); add Sentry to front-end + serverless fns.	PostHog + Sentry	2 days	0
18	Accessibility gaps	Add text labels to status (not color only); label icon buttons; run axe/Lighthouse; manual screen-reader pass on the 5 core flows.	axe (free)	1 wk	1-2
19	320px navbar clip / native controls	Make the shared navbar fluid to 320px; replace hero <code><select></code> / <code><input type=date></code> with lightweight custom components for cross-browser consistency.	CSS/JS	3 days	0-1

#	Audit issue	Solution & key steps	Free tool	Effort	Phase
20	Public repo / hotlinked images	Repo -> private; migrate supplier photos into Supabase Storage/Cloudinary with rights cleared; keep secrets in Vercel env only.	Supabase Storage	3 days	0

3.2 High-impact improvements (from audit Sec. 11) - where they land

- **Structured quote builder + real compare** (Phase 2): `quotes.line_items` as JSONB; the compare page diffs real objects.
- **Matching engine** (Phase 2): a SQL function scoring `event_type x budget x capacity x availability x location`; return top-N suppliers for an enquiry.
- **Empty states as guidance** (Phase 1): replace "no data" with next-step CTAs (cheap, high conversion impact).
- **In-app masked messaging** (Phase 2): a `messages` table + Realtime; keeps comms and audit trail on-platform (supports the anti-leakage rule).
- **SEO / indexable venue pages** (Phase 2): Astro static-generates a public page per supplier - cheapest demand channel for venue marketplaces.
- **Admin/ops console with real data** (Phase 2): reuse the `ops.html` shell, gate it behind an `admin` role + RLS, wire to real tables for verification, disputes, payouts.

4. Key integration recipes (copy-paste starting points)

4.1 Rewrite `auth.js` behind the same API (so pages don't change)

```
// auth.js - same public shape (Auth.login/getSession/getRole/requireRole/renderNav),
// real backend underneath. Pages that already call Auth.* keep working.
const Auth = {
  async login(email) {
    // passwordless OTP - no stored passwords
    return sb.auth.signInWithOtp({ email });
  },
  async getSession() {
    const { data } = await sb.auth.getSession();
    return data.session; // JWT, verified by Supabase, not localStorage
  },
  async getRole() {
    const s = await this.getSession(); if (!s) return null;
    const { data } = await sb.from('profiles').select('role').eq('id', s.user.id).single();
    return data?.role ?? null;
  },
  async requireRole(role, redirect) {
    // still a guard, but data is RLS-protected regardless
    const r = await this.getRole();
    if (r !== role) location.replace(redirect || 'signin.html');
  },
  async logout(to) { await sb.auth.signOut(); location.href = to || 'index.html'; },
  renderNav() { /* unchanged: reads getSession(), draws the dropdown */ }
};
window.Auth = Auth;
```

Because the guard is now backed by **RLS**, even if someone fakes the client state, the database returns nothing for the wrong user. The client guard becomes *cosmetic* (good UX), not *the* security boundary.

4.2 Replace mock search with a real query

```
// was: static list in search.html. now: live, filtered, availability-aware.
const { data: suppliers } = await sb.rpc('search_suppliers', {
  p_event_type: 'conference', p_min_capacity: 200,
  p_budget_band: '10-15L', p_date: '2026-09-12'
});
renderSupplierCards(suppliers); // reuse the existing card markup/CSS
```

4.3 A real payment (escrow) via a serverless function

```
// api/create-order.js (Vercel) - creates a Razorpay Route order that splits to the supplier
import Razorpay from 'razorpay';
export default async function handler(req, res) {
  const rp = new Razorpay({ key_id: process.env.RZP_KEY, key_secret: process.env.RZP_SECRET });
  const order = await rp.orders.create({
    amount: req.body.deposit * 100, currency: 'INR',
    transfers: [{ account: req.body.supplierAccountId,
      amount: req.body.deposit * 100, on_hold: true }] // held in escrow
  });
  res.json(order); // front-end opens Razorpay Checkout with this order
}
// api/rzp-webhook.js verifies signature, flips bookings.status, writes the ledger, notifies both sides
```

5. Phased roadmap (what, when, cost)

Phase	Goal	Build	Tools (all free tier)	Time	Cost
0 - Housekeeping	De-risk & instrument	Private repo; PostHog + Sentry; self-host images; fix 320px navbar; Astro migration of chrome + hashed assets	Astro, PostHog, Sentry, Supabase Storage	~1-2 wk	Rs 0
1 - Real spine (MVP)	One real booking, end to end	Supabase schema + RLS; auth.js -> Supabase Auth; persist brief/profile/settings; wire the 5 golden actions; one live escrow payment for one pilot venue	Supabase, Razorpay (test->live), Vercel fns	~4-6 wk	Rs 0 + ~2%/txn
2 - Trust & liquidity	A functioning two-sided market	Real search+matching+availability; verified reviews; KYC/verification; WhatsApp+email notifications; indexable venue pages; ops console on real data	+ Resend, WhatsApp Cloud API, Postgres FTS	~8-10 wk	~Rs 0-1k/mo
3 - Scale	Prove economics, then expand	Analytics-driven funnel work; dispute ops that meet SLAs; city-launch playbook; consider adjacent vertical; harden security & compliance audit	+ managed search / paid tiers as volume demands	6-18 mo	scales with GMV

Concrete first-90-days target (matches the audit's investment gate): 10 *real* bookings, real money through escrow, measured take-rate and quote-to-book conversion, on Rs 0 fixed tooling cost.

6. Security hardening checklist (Phase 1, mostly free)

- [] Move auth to **Supabase Auth** (server-verified JWT); delete demo credentials from source.

- [] **RLS on every table**; default-deny; policies per role. This is the real authorization.
- [] Secrets only in **Vercel env vars**; repo **private**; rotate the Gemini key (was in a public repo).
- [] **Verify Razorpay webhook signatures**; never trust client-reported payment success.
- [] Server-side **input validation** in serverless fns (never trust the browser); parametric queries only (Supabase handles this).
- [] Rate-limit auth + payment endpoints (Vercel middleware / Upstash free Redis).
- [] Privacy: keep customer contact **masked until booking** (enforce in RLS/views, not just UI); add a basic privacy policy + consent.
- [] Add **Sentry** so exceptions in the wild are visible.

7. Cost summary

Stage	Monthly fixed cost	Variable
Prototype -> Pilot (Phases 0-1)	Rs 0	~2% only when real money moves
Early commercial (Phase 2, ~200 bookings/mo)	~Rs 0-2,000 (most tools still free tier)	~2% payments, passed through take-rate
Scale (Phase 3)	Grows with GMV; still dominated by payment fees, not tooling	-

Takeaway: money is **not** the blocker to making Eventara real - focused engineering time is. The free tiers above comfortably carry a single-city pilot to its first hundreds of bookings before any meaningful bill appears, and by then the take-rate on real GMV is funding the tooling.

8. What to do Monday morning

1. Create the Supabase project and paste in the schema from Sec. 2 (half a day).
2. Make the GitHub repo private; rotate the Gemini key; add PostHog + Sentry snippets (half a day).
3. Rewrite `auth.js` internals to Supabase Auth, keeping the public API (Sec. 4.1) - now logins are real and RLS-guarded.
4. Pick **one** pilot venue; wire its enquiry -> quote -> accept -> deposit(escrow) -> confirm flow to real tables and one Razorpay test payment.
5. Migrate the shared chrome to Astro components and switch on hashed assets - the manual `?v=N` chore disappears forever.

Do these five and Eventara stops being a beautiful menu and starts being a kitchen - at essentially zero fixed cost.